# Report Mathew Exam 1: MLP Image Classification

Ingri Katherine Quevedo Rocha

M.S. Applied Economics

GWID G24132965

## Summary

My final model is an ensemble of plain MLPs (Dense, BatchNorm, ReLU and Dropout only) that reaches a local test score $(F1 + \varkappa)/2$ of **0.908** and an accuracy of **0.923**. A large improvement from my first-day model score of 0.391.

The largest single gain ($0.497 \rightarrow 0.847$) came from changing what the MLP sees, not the network depth or wide structure: color and texture histograms instead of 120,000 raw pixels. The rest came from generalization methods: data augmentation, test-time augmentation, k-fold ensembling, mix-up and label smoothing. Each change below was driven by a diagnosis of the earlier run, mostly the gap between training and validation accuracy.

## 1. Problem, data and how progress was measured

The dataset has 14,685 JPEG photos of leaves (mostly 4000 × 2672 px) in six classes. The Excel file splits them into 12,093 "train" and 2,592 "test" images. Classes are imbalanced: in the test split, class4 has 178 images and class6 has 724.

Settings were chosen on a stratified 20% validation split of the "train" rows. During the last competition day, I included in the models 5 and 15-fold cross-validation over the "train" rows, so every training image also gets an out-of-fold prediction.

The use of each metric had different conclusions:

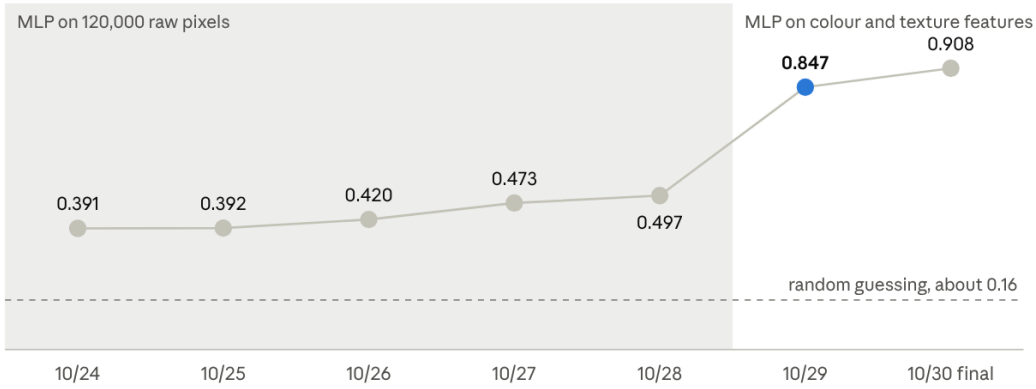
- **Accuracy** flatters the large classes: class 3 and class 6 are 55% of the test split.
- **Macro F1** weighs each class equally, so the weak class (class1) shows up.
- **Cohen's κ** corrects for agreement expected by chance (Cohen, 1960).
- **Validation loss** was a poor stopping signal for this score. In the final day run, the lowest loss was at epoch 23 while the score kept improving until epoch 51 (Figure 2), so I switched early stopping to the score.

## 2. How the model improved, day by day

Figure 1 shows the score of each day changing overtime. Five days of work on the pixel MLP added 0.11; meanwhile one change of input (10/29) added 0.35. Table 1 lists what changed and why.

**Figure 1. Switching from pixels to features moved the score from 0.50 to 0.85**

Local test score (F1 + kappa)/2 of each daily submission on 2,592 test images




Source: Claude AI generated using metrics from author's outputs.

**Table 1. Daily submissions**. Local test score = (F1 + $\varkappa$)/2 on the 2,592 "test" images.

| Submission | What changed | Why | Local test | Leaderboard |
|---|---|---|---|---|
| 10/24 | Baseline from train_example.py: 200 × 200 × 3 pixels (120,000 inputs), 6 hidden layers [300, 200, 100, 100, 80, 50], z-score inside the model, RMSprop, best epoch chosen on validation | Get a valid pipeline and an honest validation split | 0.391 | Rule Violations: Your Summary File is not an exact match with the summary generated from the model |
| 10/25 | Same network as 10/24; test script now writes the summary from the loaded model, as the grader loads it | The 10/24 summary file did not match the submitted model | 0.392 | 0.399971806065794 |
| 10/26 | Adam, BatchNorm, Kaiming init, gradient clipping, min-max [−1, 1], 10 narrower layers (15.4M weights instead of 36.1M) | Adam on 120,000 inputs blew up without BatchNorm | 0.420 | 0.419990783670582 |
| 10/27 | Geometric augmentation: flips, rotation ±20°, zoom and shift 10% | Train accuracy 0.93 vs validation 0.45: memorization | 0.473 | 0.468930642480495 |
| 10/28 | Early stopping as in NND2 Fig. 13.4, learning rate halved on plateau, balanced class weights, flip test-time augmentation | Stop at minimum validation error; help small classes | 0.497 | 0.489667282219833 |
| 10/29 | **Feature MLP**: 904 color and texture histogram features; 12-network grid → [1024, 512, 256], dropout 0.4, L2 1e-4 | Pixel MLP plateaued near 0.50 (no position invariance) | 0.847 | 0.850931242301978 |

| Submission | What changed | Why | Local test | Leaderboard |
|---|---|---|---|---|
| 10/30 | 15-model diverse ensemble, mix-up, label smoothing, early stop on score, spread, spot and metadata features | Train 0.996 vs validation 0.875; capacity made no difference | 0.908 | In the submission the file summary_Matthew.txt was mistakenly not attached. |

**Phase 1: an MLP on raw pixels (10/24–10/28)**

**10/24 reading the templates before training anything.** The first day of the competition I focused on understanding and fixing bugs of `train_example.py` which only had 1 epoch with batch size 2 on raw 0–255 pixels, no validation set and kept the epoch with the best *training* accuracy. I also fixed the bugs of `test_to_do.p`, image loading was empty, labels were never produced, and the metrics function was called without arguments.

My first version fixed the bugs and added:

- A stratified 80/20 validation split of the "train" rows with a fixed seed, so small classes are represented and runs are comparable.
- Four input normalizations: none, ÷255, and the book's min-max and z-score (Hagan & Jafari, 2026, Ch. 4).
- Switches for Dropout, BatchNorm and weight decay, the best epoch chosen by validation macro F1, and a CSV log of every run.

The "test" rows are only ever predicted, never written to or trained on. **Diagnosis**: training accuracy reached 0.93 while validation stayed at 0.45, and the validation loss climbed to 3–5. The network was memorizing.

**10/25: a valid submission first.** The 10/24 summary file did not match the submitted model. It was written before training, so it lacked the optimizer parameters. I fixed the test script now to write the summary from the loaded model.

**10/25: stabilizing training**. Tests showed that Adam with $\alpha = 0.001$ and no BatchNorm diverges on 120,000 inputs. Adam moves every weight by about $\alpha$ per step whatever the gradient, so each first-layer net input jumps by roughly $\alpha \times 120{,}000$, and gradient clipping cannot stop it. BatchNorm re-normalizes the net input of every layer each batch and took the loss from 2.2 to 0.7, stable (Ioffe & Szegedy, 2015). I also used:

- Adam with the Kingma–Ba defaults $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, e = 1e-8 (Kingma & Ba, 2014; Hagan & Jafari, 2026, Eq. 3.29–3.33).
- Kaiming initialization, weight variance 2/S^(m−1), because the layers use ReLU (He et al., 2015; Hagan & Jafari, 2026, Eq. 4.34).
- Gradient clipping at a global norm of 1.0 (Pascanu et al., 2013) and a stop on any NaN loss.

- A deeper, narrower network. A Dense layer has S^(m−1) × S^m weights and the 120,000-input first layer dominates, so 10 narrow layers cost 15.4M weights instead of 36.1M (Hagan & Jafari, 2026).
- BatchNorm without Dropout (Srivastava et al., 2014), because the book warns that the two conflict (Li et al., 2018; Hagan & Jafari, 2026, Ch. 4).
- Faster, cleaner input: JPEGs decoded directly at 1/8 size (about 2× faster), antialiased resizing (3× less resizing error on the real photos), and a RAM cache (5.8 GB of the instance's 16 GB) so each photo is decoded once. The preparation is written into the model's name, so the test script repeats it exactly.

**10/26 augmentation**. The 10/25 run had training macro F1 0.80 against validation 0.46: overfitting. To improve the generalization, I implement a geometric data augmentation as shown in Figure 2:

- Left-right and up-down flips, because the leaves are photographed at every angle.
- Rotation of at most ±20°, never 90°. The 4000 × 2672 photos are squashed to 200 × 200, and a 90° turn would put the squashing on the other axis, unlike any test image.
- Zoom and shift of at most ±10%, so spots near the edge of a leaf are not cut off.
- Mirror filling at the borders, not black. An MLP has one weight per pixel and could learn black corners as a feature.

**Figure 2.** Augmentation preview before training.



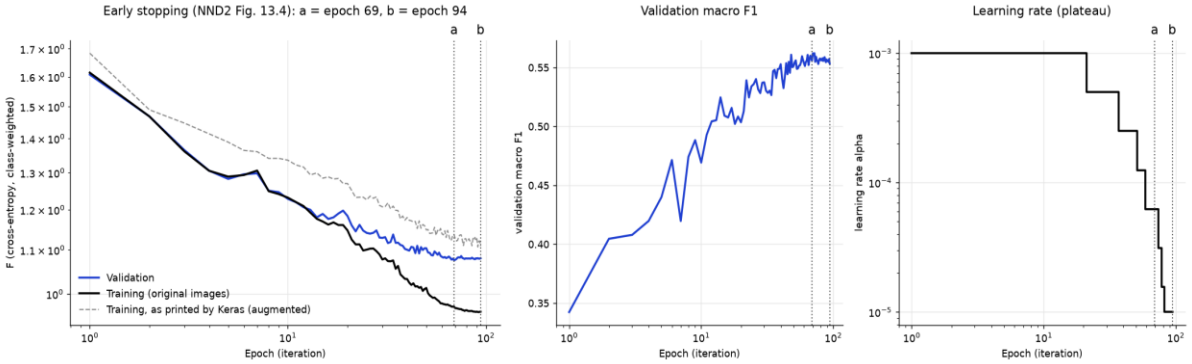
Source: Author's creation using the first five images from the dataset. Note: each row is one sample leaf, first the original, then five random versions. Every leaf stays whole and in frame.

All four are combined into one transform per image, so each image is resampled only once (less blur). The augmentation runs after the RAM cache, so every epoch sees new versions while the normalization statistics still come from the original images. I checked it against an independent SciPy implementation (within 2e-6) before using it. The result was 0.473.

**10/27 early stopping as in the book**. The old rule (patience 10 on validation F1) had stopped at epoch 32 by chance, before reaching point "a" of NND2 Fig. 13.4 (Hagan et al., 2014). With augmentation, the training loss was now above the validation loss, so the model was no longer overfitting but under-trained. I made four changes:

1. Early stopping on the validation cross-entropy with patience 25, keeping the weights of point "a", the minimum of the validation error.
2. Halving the learning rate after 4 epochs without improvement, following the book's advice to "use a relatively slow training method" near the minimum.
3. Balanced class weights N/(K·n_c), aimed at the weakest classes, class2 and class4.
4. Test-time augmentation: the prediction averages the image and its flips. The flips are written into the model's name, so the test script uses the same ones.

**Figure 2**. Implementation of early stopping to improve generalization




Source: author's calculations. Model: MLP with 10 hidden ReLU layers and BathchNorm, min-max input normalization, trained with Adam (gradients by backpropagation). Left: the validation error F reaches its minimum at point "a" (epoch 69) while the training error keeps falling until the stop at "b". Centre: the validation score $(F1 + \varkappa)/2$ keeps improving after "a" (epoch 94). Right: the learning rate is halved at each plateau of the validation error.

**10/28 test of BatchNorm**. I removed BatchNorm, set Dropout to 0.2 and doubled every layer. Both runs collapsed: the loss stayed at $\ln(6) = 1.79$ and every image was predicted as class4 (F1 0.021). Without BatchNorm, Adam at 0.001 was unstable again, as the 10/25 tests had predicted.

After reviewing the metrics and the model, I moved from tuning to improve the data augmentation. Since training and validation accuracy were 0.59 and 0.55: the model was simply limited. A Dense layer has one weight per pixel position. A lesion that moves a few pixels hits different weights, so the network would have to learn every lesion at every position from 12,000 images.

## Phase 2: describe the image, not its pixels (10/29)

The idea that moved the score from 0.497 to 0.847 was to feed the MLP histograms of color and texture over the whole leaf. A histogram counts a lesion the same wherever it sits, which is exactly

what a Dense layer cannot learn from pixels. The model itself stays a pure MLP: Input → Normalization → Dense layers → softmax.

**Table 2.** First feature set (904 values per image). Each JPEG is decoded at 1/8 size (500 × 334).

| Group | What it measures | Size | Why it helps |
|---|---|---|---|
| hs | Hue × saturation histogram | 288 | Lesion colors vs healthy green [Swain & Ballard, 1991] |
| v | Brightness histogram | 16 | Light and dark spots |
| mom | Mean and std of R, G, B, S, V + mean hue angle | 12 | Overall color cast |
| exr | Excess red (R − G) and excess green (2G − R − B) | 64 | Brown and orange lesions vs healthy tissue |
| lbp, lbp2, lbp4 | Local binary pattern histograms at 3 scales | 153 | Spot and scab texture (Ojala et al., 2002) |
| grad | Edge-strength histogram | 16 | Borders of spots and lesions |
| ctr | hs + v + lbp of the central half | 355 | Mostly leaves, little background |

Histograms are square-rooted (Hellinger normalization), so rare lesion colors are not swamped by the green majority (Arandjelović & Zisserman, 2012). The z-score of the features is the first layer of the model, computed on the training part only, with each standard deviation floored at 0.01 so an almost-constant feature cannot explode on a new image.

**Feature cache**. The features of all 14,685 images are computed once, in parallel with all CPU cores, and saved to disk. A training run then starts in seconds instead of decoding 4000 × 2672 JPEGs every epoch. This made a real grid search possible.

**Testing my over-regularization hypothesis with a grid**. My pixel logs suggested the network was over-regularized, so my first idea was more layers. The features changed the question: with 904 inputs instead of 120,000, wide layers are cheap. Instead of guessing, I made the grid answer it. I trained 12 networks: widths [512, 256], [1024, 512, 256] and [2048, 1024, 512, 256], dropout 0.2 or 0.4, L2 0 or 1e-4. All 12 scored between 0.843 and 0.853 on validation. The 0.6M, 1.6M and 4.6M-weight networks were equal, and all reached training accuracy ≥ 0.99. The best, [1024, 512, 256] with dropout 0.4 and L2 1e-4, scored 0.847 on the local test and 0.851 on the leaderboard.

**Two lessons came out of this phase**: The bottleneck had been the representation, not the capacity of the network. And on 904 normalized features, BatchNorm and Dropout together worked well, unlike on pixels.

### Phase 3: closing the generalization gap (10/30)

In order to increase the metrics, I noticed that class1 was the weakest class (F1 0.74), and class3 ↔ class6 was the largest confusion. The grid had already given the diagnosis: training accuracy

0.996, validation 0.875, and three network sizes all equal. This led me to the conclusion that the model needed more varied training data, more informative features, and lower variance in its predictions, not more capacity.

1. **K-fold ensemble**: The "train" rows are split into 5 stratified folds. Model k trains on the other 4 folds and is early stopped on fold k, so every training image is now used by 4 of the 5 models; before, the 20% validation part never trained anything. The test script averages the log-probabilities of the models (the geometric mean of their softmax outputs). The ensemble scored 0.858 on test, up from 0.847.

I also tuned one bias per class on the out-of-fold predictions and folded it into the output layer. This raised the out-of-fold score from 0.8528 to 0.8535 but lowered test from 0.8589 to 0.8581: it was only fitting the images it was tuned on, so I turned it off.

2. **Augmentation in feature space**: Each training image gets 4 random views, computed once and cached: a random crop of 60–100% of its height and width, brightness ±0.08, contrast ±15% and saturation ±10%. Hue is never changed, because color is the main signal. In every epoch each training image is shown as one of its 5 versions, picked at random; validation images are never augmented. At prediction time, each model averages its log-probabilities over the whole image and its 4 corner crops of 80%. I ranked the grid by the validation score *with* the crops, because ranking without them undervalued the augmented settings. I also added two feature groups:

- `lab` (272): CIELAB histograms, where brown, orange and yellow lesions sit further from healthy green than in HSV (CIE, 2004).
- `tile` (403): the histograms of each tile of a 4 × 4 grid, keeping for every bin the maximum over the tiles, so a lesion concentrated in one corner is not diluted by the rest of the leaf (Lazebnik et al., 2006; Boureau et al., 2010).

Augmentation with crops gave about +0.01 to +0.02 for every feature set, for example 0.848 → 0.867 for the 904 features; `lab` and `tile` added little. The 5-fold ensemble of the best combination scored **0.883** on test. Its weak spot stayed clear: class1 had recall 0.75 and F1 0.80, against 0.90–0.93 for every other class.

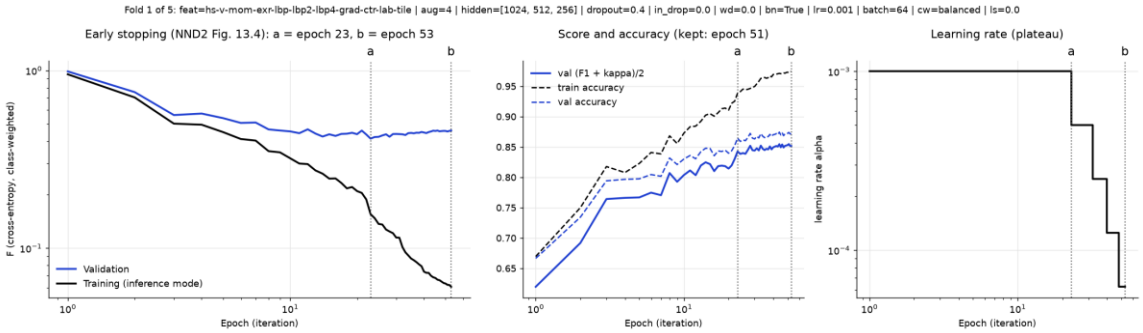
An added attempt to train the model included the following adjustments:

1. 8 augmented views instead of 4, to vary the training data further.
2. Mix-up on the feature vectors ($\alpha = 0.2$, $\lambda \geq 0.5$) and label smoothing 0.1, so the network trains on blends of two images with blended targets and cannot simply memorize them (Zhang et al., 2018; Szegedy et al., 2016).
3. A new `spread` group (98) aimed at class 1: for 48 coarse Lab color "words", the share of a 6 × 6 tile grid where each appears, plus the entropy and the number of colors present. A leaf with several lesion types has more colors, spread more widely.
4. A higher decoding resolution (1000 × 668, "ratio 4") as a grid option, so small spots are twice as large.

5. A diverse ensemble: the 3 best grid combinations (different feature sets and resolutions) were each trained on the 5 folds, and a candidate joined only if it raised the OOF score (Caruana et al., 2004). 15 models in total.

The results showed that class 1 F1 stayed at 0.78. Of 240 class1 test images, 24 were predicted as class5, 16 as class2 and 15 as class6. The remaining error was no longer a variance problem; it sat in one class.

**Figure 3.** Model with 8 augmented views.




Source: author's calculations. Model: MLP with 10 hidden ReLU layers and BathchNorm, min-max input normalization, trained with Adam (gradients by backpropagation). Left: the validation error F reaches its minimum at point "a" (epoch 23) while the training error keeps falling until the stop at "b" (epoch 53). Centre: the validation score (F1 + $\varkappa$)/2 keeps improving after "a", and the epoch kept is 51. Right: the learning rate is halved at each plateau of the validation error.

## 3. Final model

Looking at the misclassified images, I found two things. Class 1 leaves often carry several lesion types at once (small round spots together with scab blotches or rust spots). And class 5 photos mostly come from other cameras: 16 of 20 class5 test images I checked are not 4000 × 2672, against 0 of 20 for classes 2, 3 and 4. Two feature groups follow from this:

- `blob` (90): a difference-of-Gaussians spot detector on the L*, a* and b* channels at 3 spot sizes. It measures how much and how many light, dark, red-brown and yellow-orange spots a leaf has [Lindeberg, 1998].
- `meta` (5): image size, aspect ratio and JPEG bytes per pixel, read from the file header.

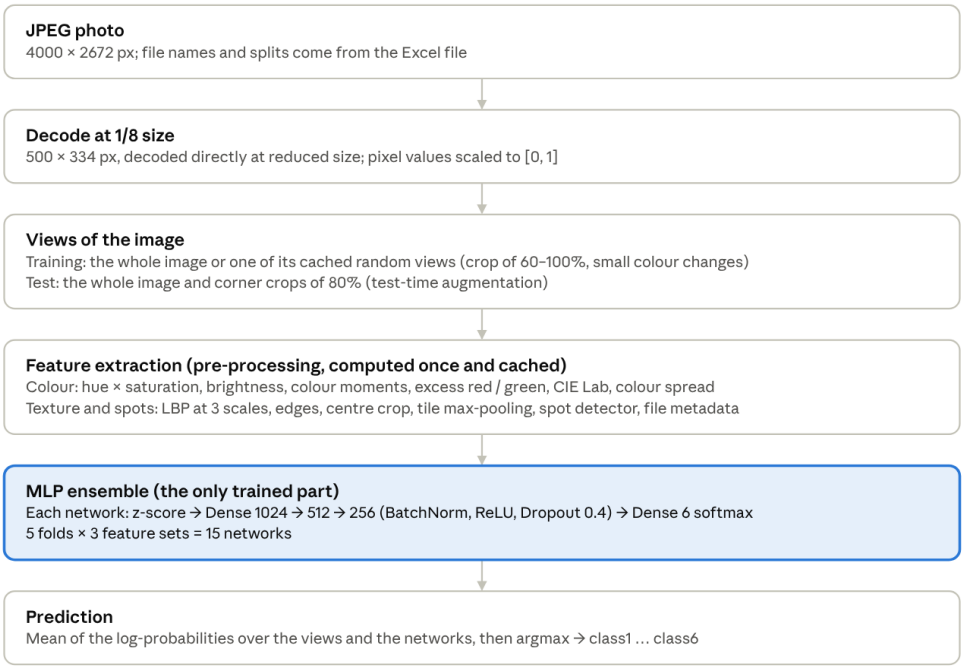
`meta` describes how the photos were taken, not the leaf, so it is a bias of this dataset. It uses only the data given, so it is allowed, but it would fail if the private set came from different cameras. The grid tests each addition on its own (7 combinations, two of them at the higher ratio-4 resolution), and only the validation and OOF scores decide whether it stays.

**Result**: the best combination was the 904 base features + spread + blob + meta (1,097 features, 1/8 resolution, 4 augmented views), with a validation score of 0.897 with test-time crops, against 0.880 for base + spread. The spot detector alone added 0.011 and the metadata alone 0.003; both ratio-4 combinations scored lower (0.884 and 0.877). The ensemble kept the 3 best combinations

(the best one, the same plus lab + tile, and base + spread + blob): 15 models with an OOF score of 0.897 (0.8945 → 0.8971 → 0.8973). The local test score is 0.908 (F1 0.913, $\varkappa$ 0.902, accuracy 0.923), and class1 F1 rose from 0.78 to 0.82 (recall 0.79). Because the OOF score was above 0.884, this run was submitted as the final model.

The final submission is an ensemble of identical-shape MLPs that differ in their feature set, their fold and their seed. Figure 4 shows the whole path from a JPEG to a class.

**Figure 4** final pipeline of train_Matthew.py and test_Matthew.py




Source: Claude AI generated using metrics from author's outputs.

**Table 3.** Final network design

| Layer | Output | Trainable weights (d = 1,097) |
|---|---|---|
| Input (features) | d | 0 |
| Normalization: z-score with training mean and std | d | 0 |
| Dense 1024, no bias → BatchNorm → ReLU → Dropout 0.4 | 1024 | 1,123,328 + 4,096 |
| Dense 512, no bias → BatchNorm → ReLU → Dropout 0.4 | 512 | 524,288 + 2,048 |
| Dense 256, no bias → BatchNorm → ReLU → Dropout 0.4 | 256 | 131,072 + 1,024 |
| Dense 6, softmax | 6 | 1,542 |
| Total | | 1,787,398 |

One network of ensembles: d = number of features: 1,097 for the best set (base + spread + blob + meta). The other two sets of the ensemble have 1,772 and 1,092 features (2,478,598 and 1,782,278 weights).. The Dense layers before BatchNorm have no bias, because BatchNorm subtracts the mean and adds its own shift.

**Table 4.** Final network training settings.

| Setting | Value | Reason |
|---|---|---|
| Optimizer | Adam, $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, e = 1e-8 | Kingma–Ba defaults |
| Batch size | 64 | |
| Gradient clipping | global norm 1.0 | Tames single bad batches |
| Loss | categorical cross-entropy, label smoothing 0.1 | Less over-confident targets |
| Class weights | balanced, N/(K·n_c) | Small classes count as much as large ones |
| Mix-up | $\alpha = 0.2$, $\lambda \geq 0.5$ | Less memorization |
| Learning rate | × 0.5 after 8 epochs without a better validation loss, min 1e-6 | Slow down near the minimum |
| Early stopping | patience 20 on validation (F1 + ϰ)/2, max 300 epochs; weights of the best epoch kept | Stop where the leaderboard metric peaks |
| Augmentation | 4 cached random views per image | More varied training data |
| Test-time augmentation | 5 views: whole image + 4 corner crops of 80% | Lower variance per image |
| Ensemble | 5 folds × 3 feature sets = 15 networks, average of log-probabilities | Lower variance across models |

## 4. What did not work, and what I learned

**Table 5.** Ideas that did not help.

| Idea | Result | Decision |
|---|---|---|
| Wider pixel network, Dropout 0.2, no BatchNorm | Collapsed to one class, F1 0.02 | BatchNorm kept for pixels |
| Early stopping on validation F1, patience 10 (pixels) | Stopped at epoch 32 while validation error still fell | Stop on validation error (NND2 Ch. 13) |
| 16 × 16 color thumbnail as a feature | Lowered the 240-image pilot score | Dropped |
| Bigger networks (4.6M vs 0.6M weights) | Same validation score, about 0.85 | Capacity is not the bottleneck |
| Per-class output bias tuned out-of-fold | Out-of-fold +0.0007, test −0.0008 | Turned off |
| 8 augmented views instead of 4 | OOF −0.007 in the grid | Back to 4 in the final run |

| Idea | Result | Decision |
|---|---|---|
| 15-model diverse ensemble vs 5 models | Out-of-fold 0.868 → 0.884, test 0.883 → 0.883 | Kept, but no claim of a test gain |
| Mix-up on the feature vectors | OOF −0.001, within noise | Kept at $\alpha = 0.2$ in the final run (difference within noise) |
| Removing BatchNorm, Dropout 0.2, doubled layers (pixels, 10/27) | Loss stuck at $\ln(6) = 1.79$, every image predicted as class4 | Keep BatchNorm and Dropout simultaneously |

**Lessons.**

1. **Diagnose before changing.** The gap between training and validation accuracy, and whether network size mattered, decided every next step: stabilize, then regularize, then change the input, then reduce variance.
2. **The input matters more than the network for an MLP**. Position-invariant pre-processing, histograms gave the MLP a bump on its generalization.
3. **Out-of-fold gains are not always test gains**. The bias tuning and the 15-model ensemble both looked better on the data used to choose them. A held-out test split, never used to choose, kept the evaluation honest.
4. **The book's methods help with regularization.** Early stopping at point "a", a slow learning rate near the minimum, Kaiming initialization and normalization inside the model all came from the NND2 book (Hagan & Jafari, 2026).

## References


Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. OSDI, 265–283.

Anthropic. (2026). Claude (Opus 5.5) [Large language model]. https://claude.ai. Used as an assistant for code review, writing and testing parts of train_Matthew.py and test_Matthew.py, and drafting this report, October 24–30, 2026.

Arandjelović, R., & Zisserman, A. (2012). Three things everyone should know to improve object retrieval. *CVPR*, 2911–2918.

Boureau, Y.-L., Ponce, J., & LeCun, Y. (2010). A theoretical analysis of feature pooling in visual recognition. *ICML*, 111–118.

Caruana, R., Niculescu-Mizil, A., Crew, G., & Ksikes, A. (2004). Ensemble selection from libraries of models. *ICML*.

CIE (2004). *Colorimetry* (3rd ed.), CIE 15:2004. Commission Internationale de l'Éclairage.

Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37–46.

Hagan, M. T., Demuth, H. B., Beale, M. H., & De Jesús, O. (2014). Neural Network Design (2nd ed.). Martin Hagan. Ch. 13, "Generalization": early stopping (Fig. 13.4), regularization. Cited as NND2.

Hagan, M. T., & Jafari, A. H. (2026). Neural Network Design: Deep Learning (draft, January 2026). Ch. 3: Adam (Eq. 3.29–3.33); Ch. 4: input normalization, batch normalization, Kaiming initialization (Eq. 4.34), gradient clipping, dropout, L2 regularization.

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. *ICCV*, 1026–1034.

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. arXiv:1502.03167.

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv:1412.6980.

Lazebnik, S., Schmid, C., & Ponce, J. (2006). Beyond bags of features: Spatial pyramid matching for recognizing natural scene categories. *CVPR*, 2169–2178.

Li, X., Chen, S., Hu, X., & Yang, J. (2018). Understanding the disharmony between dropout and batch normalization by variance shift. arXiv:1801.05134 (CVPR 2019).

Lindeberg, T. (1998). Feature detection with automatic scale selection. *International Journal of Computer Vision*, 30(2), 79–116.

Ojala, T., Pietikäinen, M., & Mäenpää, T. (2002). Multiresolution grayscale and rotation invariant texture classification with local binary patterns. *IEEE TPAMI*, 24(7), 971–987.

Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the difficulty of training recurrent neural networks. *ICML*, 1310–1318.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. JMLR, 12, 2825–2830. Used for the stratified splits and the metrics.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *JMLR*, 15, 1929–1958.

Swain, M. J., & Ballard, D. H. (1991). Color indexing. *International Journal of Computer Vision*, 7(1), 11–32.

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the Inception architecture for computer vision. *CVPR*, 2818–2826.

train_example.py and test_to_do.py, the sample code provided with Exam 1, used as the base of train_Matthew.py and test_Matthew.py.

Zhang, H., Cisse, M., Dauphin, Y. N., & Lopez-Paz, D. (2018). mix-up: Beyond empirical risk minimization. *ICLR*.